

Contents

guide_Efficient Memory Management for Large Language Model Serving with PagedAttention

0. 这篇论文到底解决什么问题

1. 回到 2023 年的语境

2. 先把 serving 流程读清楚

3. 传统连续 KV cache 为什么浪费

4. PagedAttention 的核心抽象

5. 张量级别怎么落地

6. attention 数学有没有变

7. vLLM 系统结构

8. decoding 生命周期

9. prefix sharing 和 copy-on-write

10. block size 的设计理由

11. 本地代码怎么对应论文

12. 实验证据链

13. 这篇论文的限制和代价

14. 对今天的意义

15. 常见误解

16. 学习时应该掌握的最小闭环

17. 用 AI agent 正确学习这篇

18. 读论文原文时的路线

1

1

2

2

3

3

4

6

6

7

8

8

10

10

11

11

11

11

12

guide_Efficient Memory Management for Large Language Model Serving with PagedAttention

原论文: Efficient Memory Management for Large Language Model Serving with PagedAttention
作者: Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, Ion Stoica
会议和年份: SOSP 2023
本地原文 PDF: learning/inference-engine-core/paper/01_vllm_pagedattention.pdf
本地代码入口:
learning/inference-engine-core/src/paged_kv.py
learning/inference-engine-core/src/continuous_batching.py
learning/inference-engine-core/src/naive_kv.py

0. 这篇论文到底解决什么问题

这篇论文的主角不是一个新的大模型结构，也不是一个改变注意力数学意义的近似算法。它解决的是 LLM serving 里很朴素、也很要命的工程问题: 自回归生成时，KV cache 太大，而且每个请求的 KV cache 长度动态增长。如果系统像传统张量一样为每个请求预留一整段连续显存，就会把大量显存浪费在尚未生成的 token、最大长度预留、内存碎片和重复拷贝上。

vLLM 的核心洞察是: KV cache 可以像操作系统管理虚拟内存那样管理。论文把每个请求的 KV cache 切成固定大小的 KV block，用 block table 维护“逻辑 block 到物理 block”的映射。这样，用户序列在逻辑上仍然是连续的 token 历史，但在 GPU 显存里可以是不连续的物理块。

一句话主张:

PagedAttention = 用分页式 KV cache 管理, 让 attention kernel 能读取非连续 KV block。
vLLM = 基于 PagedAttention 的高吞吐 LLM serving 系统。

它的价值链条是:

KV cache 浪费少

- > 同一张 GPU 能放下更多请求
- > decoding 阶段 batch 更大
- > GPU 利用率更高
- > 同等延迟下吞吐提高

1. 回到 2023 年的语境

2023 年的 LLM serving 已经不只是 “能不能生成文本”, 而是 “能不能便宜、稳定、高吞吐地服务大量在线请求”。大模型参数常驻显存, prefill 和 decoding 反复调用 Transformer。单个请求的输出长度不可预测, 且每个请求都需要保存历史 token 的 key/value 向量, 便于下一步生成时复用。

论文用 13B 参数模型在 NVIDIA A100 40GB 上说明压力来源:

- 模型权重约占 65% 显存, 长期常驻。
- KV cache 可占接近 30% 显存, 而且随请求数量和生成长度动态变化。
- activation 占比相对小, 短暂存在。

这意味着 serving 系统真正能调度的弹性空间, 很大一部分就在 KV cache 管理里。模型参数不能随便变, activation 生命周期短, 而 KV cache 既大又动态, 正好成为吞吐瓶颈。

论文还给出一个很能建立直觉的数字: 对 OPT-13B, 一个 token 的 KV cache 约 800 KB, 计算方式是:

$$2 \times hidden_size \times n_layers \times bytes_per_element$$

具体到 OPT-13B:

$$2 \times 5120 \times 40 \times 2\ bytes \approx 800\ KB$$

如果最大序列长度是 2048, 一个请求的 KV cache 最坏可以到约 1.6 GB。几十 GB 显存看起来很大, 但一旦每个请求都按最大长度预留, batch 很快就被压垮。

2. 先把 serving 流程读清楚

LLM 在线生成通常分成两个阶段。

第一阶段是 prefill。输入 prompt 有多个 token, 模型可以一次性处理 prompt, 计算出每个 prompt token 的 KV cache, 并产生第一个输出 token 的分布。

第二阶段是 decoding。模型每次只生成一个新 token。每一步都需要拿当前 token 的 query 去看之前所有 token 的 key/value, 所以历史 KV cache 必须保留下来。

可以把一个请求看成这样:

```
prompt tokens:    p0 p1 p2 p3 p4 p5 p6
generated tokens: y0 y1 y2 ...
```

prefill 后:

KV cache 保存 p0..p6 的 K/V

第 1 次 decode:

输入 y_0 , 读取 $p_0..p_6$ 的 K/V, 写入 y_0 的 K/V

第 2 次 decode:

输入 y_1 , 读取 $p_0..p_6, y_0$ 的 K/V, 写入 y_1 的 K/V

decoding 的计算粒度很小, 每个请求每步只前进一个 token。为了让 GPU 不闲着, 系统要把很多请求放在同一次 iteration 里一起跑。这就是 continuous batching 和 iteration-level scheduling 的背景。但是 batch 能不能变大, 取决于显存里能不能容纳更多请求的 KV cache。

所以这篇论文不是单独解决 “attention 怎么算”, 而是把 attention kernel、KV cache allocator、scheduler 绑在一起优化。

3. 传统连续 KV cache 为什么浪费

传统做法通常把每个请求的 KV cache 存成一段连续张量。问题是输出长度不可预测, 系统往往要根据最大可能长度预留空间。

浪费主要有三类。

第一类是 reserved waste: 请求当前只生成到很短的位置, 但系统已经为未来 token 预留了槽位。这些槽位未来可能会用到, 但在当前时刻占着显存, 阻止其他请求进入 batch。

第二类是 internal fragmentation: 系统按照最大长度或较大的分配粒度给请求分配空间, 请求结束后发现很多槽位永远没有用过。

第三类是 external fragmentation: 显存 allocator 中存在零散空洞, 不足以拼成某个请求需要的连续大块。

直觉图:

contiguous KV layout

Request A, max_len=2048:

[used prompt][used output][reserved future][never used]

Request B, max_len=512:

[used prompt][reserved future][never used]

GPU memory:

[Request A huge chunk][small hole][Request B chunk][hole][...]

论文的 profiling 很关键: 在已有系统中, 真正存储有效 token state 的 KV cache 内存比例只有约 20.4% 到 38.2%。也就是说, 系统以为自己没有显存了, 但其中大部分空间并没有保存当前有用的 token K/V。

vLLM 的目标不是减少每个 token 的 KV 向量大小, 而是减少 “为了放这些 KV 向量而额外浪费的显存”。

4. PagedAttention 的核心抽象

PagedAttention 借用了 OS 虚拟内存的三个概念:

- token 类似 byte。
- KV block 类似 page。
- request 的逻辑 token 序列类似进程虚拟地址空间。

每个请求维护一张 block table。逻辑上连续的 block 可以映射到任意物理 block。

Request A logical KV blocks

logical block id:	0	1	2
tokens:	0..15	16..31	32..47
block table:	7	1	3

Physical KV blocks on GPU

physical block 0: free or used by others
physical block 1: Request A logical block 1
physical block 2: free or used by others
physical block 3: Request A logical block 2
physical block 7: Request A logical block 0

这解决了连续分配的问题。请求 A 在逻辑上仍然拥有连续 token 历史，但物理显存不要求连续。一个新 token 到来时，只要最后一个 block 还有 slot，就写进去。如果最后一个 block 满了，再分配一个新的物理 block。

关键不变量:

- 只有最后一个 block 可能没填满。
- 新物理 block 只在之前 block 填满后才分配。
- 因为所有 physical block 大小相同，外部碎片基本被消除。
- 因为按需分配，未来 token 不再提前占满显存。
- 因为 block table 是间接映射，多个逻辑 block 可以共享同一个物理 block。

5. 张量级别怎么落地

在 Transformer 里，KV cache 是按 layer 保存的。一个 token 在某一层有 key 和 value。粗略形状可以写成:

K cache: [num_layers, num_tokens, num_kv_heads, head_dim]
V cache: [num_layers, num_tokens, num_kv_heads, head_dim]

PagedAttention 把 num_tokens 这一维切成固定大小 block。仓库里的教学实现 PagedKvPool 用的是:

k shape = [n_layers, n_blocks, block_size, n_kv_heads, head_dim]
v shape = [n_layers, n_blocks, block_size, n_kv_heads, head_dim]

一个 token 的逻辑位置 pos 通过两步定位:

```
logical_block = pos // block_size  
slot = pos % block_size  
physical_block = block_table[logical_block]
```

然后实际写入:

```
k[layer, physical_block, slot] = token_k  
v[layer, physical_block, slot] = token_v
```

这个映射非常重要。你读论文时不要只记“有个 block table”，而要在脑子里形成这张五维张量的图:

PagedKvPool.k

```

layer dimension
  layer 0:
    physical block 0:
      slot 0: [kv_head, head_dim]
      slot 1: [kv_head, head_dim]
      ...
    physical block 1:
      slot 0: [kv_head, head_dim]
      ...
  layer 1:
    physical block 0:
      ...

```

```

BlockTable for one request:
  logical block 0 -> physical block 7
  logical block 1 -> physical block 1
  logical block 2 -> physical block 3

```

6. attention 数学有没有变

标准自注意力对当前 query q_i 的输出是:

$$o_i = \sum_{j=1}^i \text{softmax}(q_i k_j^T / \sqrt{d})_j v_j$$

PagedAttention 不改变这个数学含义。它只是把历史 token 的 K/V 按 block 存放。令每个 block 大小为 B , 第 b 个 block 里包含一段连续 token 的 key/value:

$$K_b = (k_{(b-1)B+1}, \dots, k_{bB})$$

$$V_b = (v_{(b-1)B+1}, \dots, v_{bB})$$

kernel 做的事情是: 对 block table 里的每个 logical block, 找到 physical block, 取出其中有效 token 的 K/V, 参与同一个 softmax attention.

伪代码:

```

scores = []
values = []

for logical_block, physical_block in enumerate(block_table):
    k_block, v_block = fetch(physical_block)
    valid_tokens = trim_padding_if_last_block(k_block, v_block)
    scores.append(q @ valid_tokens.k.T / sqrt(head_dim))
    values.append(valid_tokens.v)

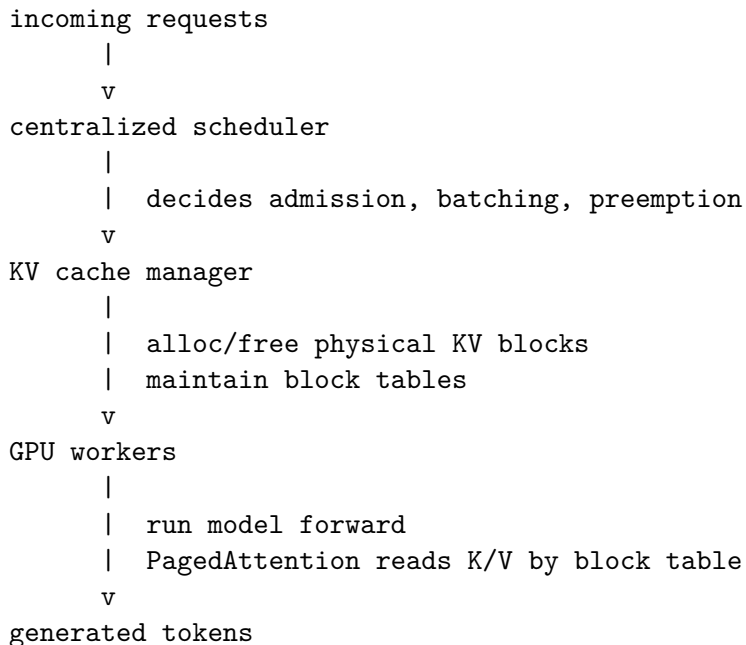
attn = softmax(concat(scores))
output = attn @ concat(values)

```

所以 PagedAttention 的难点不是公式新，而是 kernel 必须支持“按 block table 间接读取非连续显存”。这会引入额外访存和分支开销，但换来系统层面的显存效率和更大 batch。

7. vLLM 系统结构

论文里的 vLLM 由几个部分配合：



scheduler 负责决定哪些请求进入本轮 batch。KV cache manager 负责物理 block 的分配、释放、共享和引用计数。GPU worker 收到本轮输入 token 和对应 block table 后，执行模型 forward，并把新生成 token 的 KV 写进物理 block。

这也是论文很“系统”的地方：如果只有 PagedAttention kernel，没有调度器和 cache manager，系统无法把显存节省转化成吞吐。如果只有 scheduler，没有分页式 KV cache，batch 还是会被连续显存预留卡住。

8. decoding 生命周期

用一个 block size 为 4 的例子看完整过程。

prompt 有 7 个 token:

```
logical block 0: token 0 1 2 3
logical block 1: token 4 5 6 _
```

prefill 后，block table 可能是:

```
logical block 0 -> physical block 7
logical block 1 -> physical block 1
```

第一次 decode 生成 token 7。最后一个 logical block 还有一个空 slot，所以直接写入:

```
logical block 1: token 4 5 6 7
```

第二次 decode 生成 token 8。最后一个 block 已满，系统分配新物理 block:

```
logical block 2 -> physical block 3
logical block 2: token 8 _ _ _
```

这正是仓库里 `BlockTable.append_token` 的逻辑:

```
def append_token(self, layer, k, v):
    self.ensure_capacity(self.n_tokens + 1)
    blk = self.block_ids[self.n_tokens // self.block_size]
    slot = self.n_tokens % self.block_size
    self.pool.write_token(layer, blk, slot, k, v)
    if layer == 0:
        self.n_tokens += 1
```

`ensure_capacity` 决定是否需要新 block, `n_tokens // block_size` 找 logical block, `n_tokens % block_size` 找 block 内 slot.

9. prefix sharing 和 copy-on-write

PagedAttention 的另一个贡献是让 KV cache sharing 变得自然。

场景一: parallel sampling. 用户希望同一个 prompt 采样多个输出。prompt 部分完全相同, 理应共享 KV cache。传统连续布局下, 不同输出序列通常各自维护一份 KV cache, 造成重复。vLLM 让多个序列的 block table 指向同一批 physical blocks。

```
sample A block table:
    logical 0 -> physical 7
    logical 1 -> physical 1
```

```
sample B block table:
    logical 0 -> physical 7
    logical 1 -> physical 1
```

```
refcount:
    physical 7: 2
    physical 1: 2
```

当两个 sample 开始生成不同 token 时, 它们不能继续写同一个共享 block。vLLM 使用 copy-on-write:

```
before write:
    sample A logical 1 -> physical 1, refcount=2
    sample B logical 1 -> physical 1, refcount=2
```

```
sample A wants to write:
    allocate physical 3
    copy physical 1 to physical 3
    sample A logical 1 -> physical 3
    physical 1 refcount decreases to 1
```

这样只有真正要写入的最后一个 block 需要复制, 历史完整 block 可以继续共享。

仓库里的 `fork` 对应这个思想:

```
def fork(self):
    child = BlockTable(pool=self.pool, n_tokens=self.n_tokens)
    for blk in self.block_ids:
```

```

        self.pool.share_block(blk)
        child.block_ids.append(blk)
    return child

```

教学实现里 fork 展示了引用计数和 block table 共享。真正生产系统还要在写共享 block 时执行 copy-on-write 的复制和重映射。

场景二: beam search。beam candidate 之间不只共享 prompt, 还会共享中间前缀。随着 beam 扩展和裁剪, 一些 candidate 被淘汰, 对应 physical block 的 refcount 下降到 0 后释放; 新的 candidate 继承幸存路径的 block table, 并在分叉处 copy-on-write。论文强调, beam search 中 vLLM 的共享收益更明显, 因为不同候选序列常共享很长前缀。

场景三: shared prefix。很多应用有固定 system prompt 或 few-shot examples。多个请求共享长前缀时, vLLM 可以预先缓存这部分 KV, 并让不同请求的 block table 指向相同 physical blocks。

10. block size 的设计理由

block size 是 PagedAttention 的关键超参数。

太小的问题:

- block table 更长。
- kernel 读取更多小块, 间接寻址开销更高。
- 每个 block 内可并行处理的 token 少, GPU 利用率可能下降。

太大的问题:

- 最后一个 block 的空 slot 更多, 内部碎片增加。
- prefix sharing 粒度变粗, 能共享的概率下降。
- 短序列 workload 更容易被大 block 拖累。

论文的 ablation 结论是: ShareGPT 这种较长序列 workload 中, block size 16 到 128 表现较好; Alpaca 这种短序列 workload 中, 16 和 32 更合适, 大 block 会明显变差。vLLM 最终默认使用 block size 16, 因为它足够利用 GPU, 又不会制造太多内部碎片。

你可以把这个 trade-off 记成:

small block = finer memory granularity, more indexing overhead

large block = better contiguous work per block, more fragmentation and less sharing

11. 本地代码怎么对应论文

learning/inference-engine-core/src/naive_kv.py 是传统静态 KV cache 的 baseline。它的形状是:

```
[n_layers, batch, max_len, n_kv_heads, head_dim]
```

这里的 max_len 对所有 batch slot 预留, 所以很容易浪费。demo_fragmentation 用随机长度展示: 当平均实际长度远小于 max_len 时, reserved tokens 大量闲置。

learning/inference-engine-core/src/paged_kv.py 是论文核心抽象的教学实现。

对应关系:

```

PagedKvPool
- physical KV block pool
- k/v tensors
- free_ids

```


- refcount

BlockTable

- per-request logical-to-physical mapping
- block_ids
- n_tokens
- append_token
- fork

utilization

- used logical token slots / reserved block slots

最小实验代码:

```
import torch
from paged_kv import PagedKvPool, BlockTable, utilization

pool = PagedKvPool(
    n_blocks=128,
    block_size=16,
    n_kv_heads=8,
    head_dim=64,
    n_layers=2,
)

tables = [BlockTable(pool) for _ in range(4)]
lengths = [40, 17, 95, 8]

for table, length in zip(tables, lengths):
    for _ in range(length):
        k = torch.randn(8, 64, dtype=torch.float16)
        v = torch.randn(8, 64, dtype=torch.float16)
        for layer in range(2):
            table.append_token(layer, k, v)

print(pool.n_free())
print(utilization(tables))
print([len(t.block_ids) for t in tables])
```

这个例子应该重点观察两个东西:

- 长度不同的请求不再各自预留到统一最大长度。
- 浪费只来自每个请求最后一个未填满 block。

learning/inference-engine-core/src/continuous_batching.py 对应论文里的 iteration-level scheduling 思想。它不实现真实 GPU worker, 但保留了核心循环:

```
admit pending requests if KV budget allows
run one forward over running requests
append one sampled token per request
retire finished requests
repeat
```

论文里的 scheduler 更复杂，会结合 physical block 是否足够、preemption、swapping、distributed workers 等策略。但学习时先掌握这个循环，就能理解为什么“节省 KV 显存”会直接影响 admission 和 batch size。

12. 实验证据链

这篇论文的实验不是孤立地说“kernel 快”。事实上，PagedAttention kernel 本身因为 block table 间接访问，会比高度优化的 contiguous attention kernel 有额外开销。论文报告 attention kernel latency 约高 20% 到 26%。真正的系统收益来自显存管理改善后，end-to-end serving 可以 batch 更多请求。

证据链可以这样读。

第一步，先证明已有系统浪费 KV 显存。论文的 Fig. 2 显示，已有系统中实际 token state 对 KV cache 分配空间的利用率只有约 20.4% 到 38.2%，而 vLLM 接近 96.3%。这说明瓶颈确实存在，而且 PagedAttention 直接打中瓶颈。

第二步，证明更高显存利用率转化成更大 batch。OPT-13B 在 ShareGPT workload 中，vLLM 同时处理的请求数量约为 Orca Oracle 的 2.2 倍，约为 Orca Max 的 4.3 倍。

第三步，证明更大 batch 转化成同延迟下更高吞吐。ShareGPT 上，vLLM 在相似延迟下可支撑比 Orca Oracle 高 1.7 到 2.7 倍的 request rate，比 Orca Max 高 2.7 到 8 倍。相比 FasterTransformer，最高可达 22 倍 request rate。

第四步，证明复杂 decoding 更受益。parallel sampling 里，多个输出共享 prompt KV；beam search 里，不同 beam candidate 共享更长前缀。论文报告 Alpaca 上 parallel sampling 的 KV block sharing 带来约 6.1% 到 9.8% memory saving，beam search 带来约 37.6% 到 55.2%。ShareGPT 上对应更高，parallel sampling 约 16.2% 到 30.5%，beam search 约 44.3% 到 66.3%。

第五步，证明 shared prefix 场景也有效。翻译 workload 中，共享 one-shot prefix 时，vLLM 吞吐约为 Orca Oracle 的 1.67 倍；共享 5-shot prefix 时约为 3.58 倍。prefix 越长，重复 KV 越多，block sharing 越有价值。

第六步，说明收益条件。对于短序列、显存充足、系统已经 compute-bound 的场景，vLLM 相对 Orca Oracle 的优势会变小。论文没有把 PagedAttention 包装成任何情况下都免费的魔法，而是很清楚地说明它主要解决 memory-bound serving。

13. 这篇论文的限制和代价

PagedAttention 不是没有成本。

第一，attention kernel 要按 block table 间接读取 K/V，会引入额外内存访问、分支和变长处理。论文微基准中 attention kernel latency 比 FasterTransformer 的优化实现高约 20% 到 26%。

第二，系统复杂度增加。cache manager、block table、refcount、copy-on-write、preemption、swapping 都需要正确协作。任何 refcount 错误都可能造成内存泄漏、错误共享或覆盖仍被别的序列使用的 KV block。

第三，block size 需要权衡。没有一个 block size 在所有 workload 中都最优。

第四，PagedAttention 主要适合 KV cache 成为显存瓶颈的场景。如果请求很短、batch 很小、或者瓶颈主要在 compute，而不是 KV memory，那么系统收益可能不明显。

第五，它不改变模型输出质量。论文强调 vLLM 不改变模型数学结果，收益来自 serving 系统，因此不能把它理解为模型能力提升。

14. 对今天的意义

PagedAttention 后来成为现代 LLM serving 的基础概念之一。今天你看到的很多推理系统能力，比如高并发 decoding、prefix cache、continuous batching、beam/prefix sharing、chunked prefill、KV cache offload，都和“KV cache 是可调度资源”这个思想有关。

这篇论文的长期意义在于，它把 LLM serving 从“调用模型生成”推进到“围绕 KV cache 做资源管理”。它让工程师意识到，Transformer 推理不只是算子优化问题，也是操作系统式的内存管理问题。

如果你以后读 TensorRT-LLM、SGLang、vLLM 新版本、PagedAttention kernel、FlashInfer、prefix caching 或 speculative decoding serving 论文，这篇都应该作为底层坐标系。

15. 常见误解

误解一：PagedAttention 是近似 attention。

不是。它保持标准 attention 的数学结果，只改变 KV cache 的存储和访问方式。

误解二：它让单次 attention kernel 更快。

不一定。论文反而报告了 kernel 级额外开销。它让系统端到端吞吐更高，原因是能 batch 更多请求。

误解三：分页只是在 CPU/OS 里有用，GPU 上没意义。

这篇论文的贡献正是把 OS 分页思想适配到 LLM KV cache。不同点是它结合了 token 生成、attention 访问模式、prefix sharing 和 GPU kernel。

误解四：KV cache sharing 只是 prompt cache。

prompt sharing 是一个场景。beam search 中间前缀共享、parallel sampling prompt 共享、跨请求 shared prefix 都可以由 block table 和 refcount 统一表达。

误解五：vLLM 只是一套 allocator。

不是。allocator 是核心，但必须和 PagedAttention kernel、scheduler、GPU workers 一起工作。

16. 学习时应该掌握的最小闭环

读完这篇，你应该能闭卷回答下面几个问题。

1. 为什么 LLM serving 的 decoding 阶段需要 batch 很多请求？
2. 为什么 KV cache 会成为 batch size 的瓶颈？
3. 连续 KV cache 的 reserved waste、internal fragmentation、external fragmentation 分别是什么？
4. 为什么 fixed-size physical blocks 可以缓解外部碎片？
5. block table 里存的是什么？
6. 逻辑 token 位置如何转换为 physical block 和 slot？
7. PagedAttention 是否改变 attention 数学结果？
8. copy-on-write 为什么只需要复制一个 block，而不是整段 KV cache？
9. block size 太小和太大的代价分别是什么？
10. 为什么 kernel latency 变高时，end-to-end throughput 仍然可能大幅提高？

17. 用 AI agent 正确学习这篇

你可以让 agent 帮你做三轮，但每轮都要有可检查输出。

第一轮，让 agent 画图，不要让它总结：

请用 `block_size=4`、`prompt_len=7`、生成 3 个 token 的例子，逐步画出 block table、physical block、slot 写入位置和 refcount 变化。
不要省略任何一次分配。

第二轮，让 agent 对照代码：

请只基于 `learning/inference-engine-core/src/paged_kv.py`，
解释 `append_token`、`ensure_capacity`、`fork`、`utilization` 分别对应论文哪一段设计。
每个函数给一个最小输入例子。

第三轮，让 agent 反向考你：

请用闭卷考试方式问我 10 个 PagedAttention 问题。
如果我回答错了，不要直接给答案，先指出我混淆的是
KV cache 数学、block table 映射、还是 serving scheduler。

关键是不要让 agent 只生成“论文摘要”。这篇必须手画 block table，手算 slot，手跑本地 `paged_kv.py`，
否则知识不会进入脑袋。

18. 读论文原文时的路线

建议顺序：

1. 先读 Introduction，抓住“KV cache 是显存瓶颈”。
2. 读 Section 3，重点看现有系统三类浪费。
3. 读 Section 4.1 到 4.3，画出 block table translation。
4. 读 Section 4.4，理解 parallel sampling、beam search 和 shared prefix。
5. 读 Section 6 和 7，不要只看倍率，追踪“显存利用率 -> batch size -> 吞吐”的证据链。
6. 最后回到本地 `paged_kv.py`，自己跑一次不同长度请求的 utilization。

当你不用原文解释这句话时，这篇就真正进脑子了：

vLLM 的优势来自把 KV cache 从 per-request contiguous allocation
改造成 paged logical-to-physical mapping，使 serving scheduler 能用
更少浪费承载更多并发序列。